ideis
UNIWERSYTET DSW

Laboratoria, 16 maja 2026r.

Architektura Aplikacji w Pythonie

[Lab 5] Data-intensive apps - PySpark

Michał Madejski

Kraków, 2026

Agenda

- **Wprowadzenie** (Ararchitektury systemów danych, Apache Spark, PySpark, kiedy używać)
- **Architektura Spark** (architektura, główne komponenty, RDD vs. dataframe, transformations vs. Actions)
- **Setup Spark** (instalacja, JDK)
- **Basic Tutorial**
- **Zadania**

ideis
UNIWERSYTET DSW

Ararchitektury systemów danych (data-intensive apps)

Czym są systemy data-intensive?

- Aplikacje, które przetwarzają, analizują lub transportują duże wolumeny danych
- Nacisk na przepustowość, niezawodność, skalowalność
- Przykłady: systemy rekomendacyjne, analityka transakcji, strumieniowanie wideo, przetwarzanie logów

Wyzwania:

Skalowalność

Spójność i dostępność

Latency vs throughput

Odporność na błędy

Złożoność integracji

ideis
UNIWERSYTET DSW

Co to jest Apache Spark?

Apache Spark to rozproszony system przetwarzania danych typu open source, który umożliwia szybką analizę dużych zbiorów danych. Jest używany do przetwarzania wsadowego, strumieniowego, interaktywnych zapytań oraz uczenia maszynowego.

Cechy Apache Spark:

Przetwarzanie w pamięci - Spark przechowuje dane w pamięci RAM, co pozwala na wielokrotnie szybsze operacje niż w przypadku tradycyjnych dyskowych systemów.

Rozproszony charakter - Umożliwia przetwarzanie danych na klastrach komputerów, co pozwala na skalowanie i przetwarzanie ogromnych ilości danych.

Wiele języków programowania - Obsługuje języki takie jak Python, Scala, Java i R, dając elastyczność w tworzeniu aplikacji.

Bogaty ekosystem - Posiada liczne biblioteki do uczenia maszynowego (MLlib), analizy danych (SQL) i przetwarzania strumieniowego (Spark Streaming).

Interfejsy API wysokiego poziomu - Ułatwiają programistom pracę z danymi i transformacjami, niezależnie od ich poziomu doświadczenia.

Dlaczego Spark?

1. **Szybkość**

Uruchamiaj programy nawet 100 razy szybciej niż Hadoop MapReduce w pamięci lub 10 razy szybciej z dysku.

Apache Spark posiada zaawansowany silnik wykonawczy oparty na DAG (Directed Acyclic Graph), który obsługuje acykliczny przepływ danych oraz przetwarzanie w pamięci (in-memory computing).

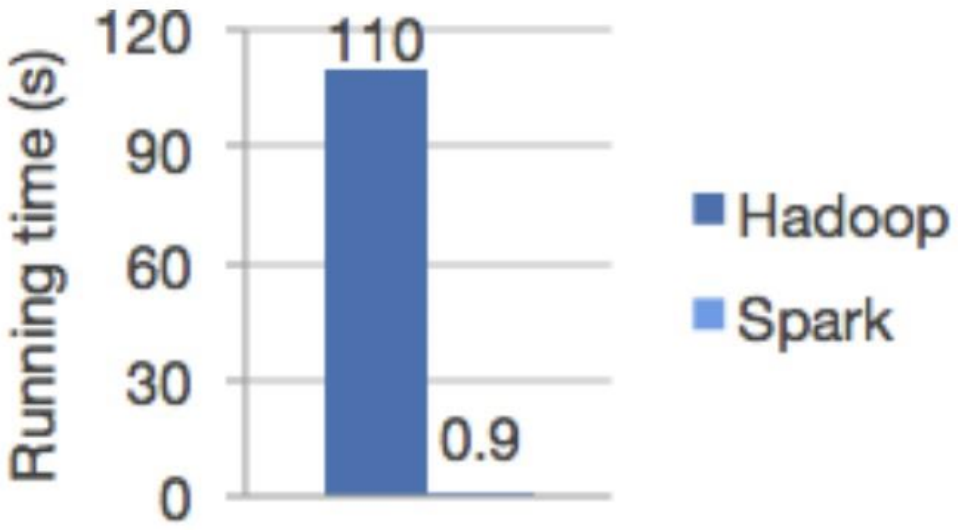


Fig. 1: Logistic regression in Hadoop and Spark

ideis UNIWERSYTET DSW

Hadoop → MapReduce → Spark

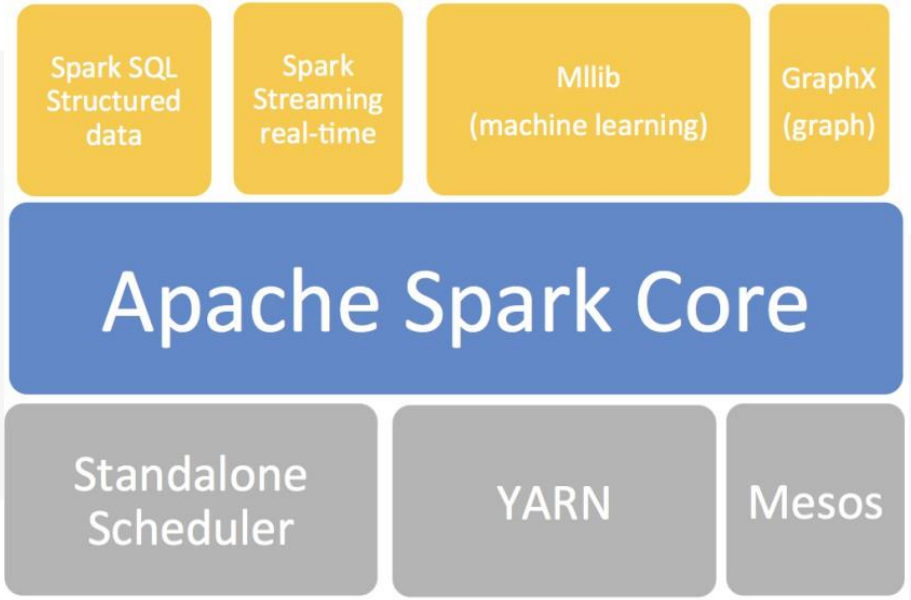
Hadoop (2006+)

- Open-source'owa platforma do **rozproszonego przechowywania i przetwarzania danych**
- Oparta na dwóch kluczowych komponentach: HDFS – system plików ; MapReduce – model programowania
- Zalety: skalowalność, odporność na awarie
- Wady: **wolne przetwarzanie** (dużo operacji dyskowych)

Apache Spark (2012+)

- Powstał na **UC Berkeley (AMP Lab)** jako szybsza alternatywa dla MapReduce
- Zamiast zapisywać dane na dysk – używa **pamięci operacyjnej (RAM)**
- Wprowadza: RDD (Resilient Distributed Dataset), DAG execution engine (graf zależności)
- 100× szybszy w przetwarzaniu wsadowym, wspiera również **strumieniowanie i ML**

Dlaczego Spark?



2. Łatwość użycia

- Twórz aplikacje szybko w Java, Scala, Pythonie lub R

3. Uniwersalność

Spark łączy SQL, przetwarzanie strumieniowe i zaawansowaną analitykę w jednym silniku

4. Działa wszędzie

Spark działa na Hadoop, Mesos, lokalnie, a także w chmurze. Obsługuje różne źródła danych: HDFS, Cassandra, HBase, Amazon S3 itd.

ideis
UNIWERSYTET DSW

PySpark = API do Sparka w Pythonie

Co to jest PySpark?

- Pythonowy interfejs do Apache Spark
- Pozwala pisać rozproszone aplikacje danych w Python
- Abstrakcja nad Spark Core — używa DataFrame API i RDD

Jak można korzystać z PySparka?

Lokalnie (standalone) – Spark uruchamiany na jednym komputerze (np. do testów, nauki)

Z poziomu notebooka (np. Jupyter, Google Colab) – Interaktywna analiza danych – Wymaga instalacji Javy + Spark

W klastrze Spark (YARN, Mesos, Kubernetes) – Produkcyjna architektura Big Data – PySpark łączy się jako klient z klastrem

Z poziomu frameworków (Airflow, Databricks, EMR) – PySpark jako krok w pipeline danych – Integracja z chmurą i orkiestratorami

ideis
UNIWERSYTET DSW

Kiedy używać PySpark

Używaj PySpark, gdy:

- Dane są zbyt duże, by zmieściły się w pamięci RAM (np. dla Pandas)
- Potrzebujesz równoległego przetwarzania na wielu rdzeniach lub maszynach
- Liczy się wydajność i skalowalność przy przetwarzaniu wsadowym
- Budujesz pipeline danych (np. ETL) łączący wiele źródeł danych
- Pracujesz z danymi w formatach rozproszonych: HDFS, Parquet, S3, bazy NoSQL
- Tworzysz aplikacje analityczne lub modele ML na dużych zbiorach danych

ideis
UNIWERSYTET DSW

Typowe zastosowania

Typowe zastosowania PySparka

Przetwarzanie dużych zbiorów danych (batch)

Budowa pipeline’ów ETL

Agregacje i analiza danych historycznych

Uczenie maszynowe na dużej skali (MLlib)

Przetwarzanie strumieni danych (Spark Streaming)

ideis
UNIWERSYTET DSW

Architektura Spark

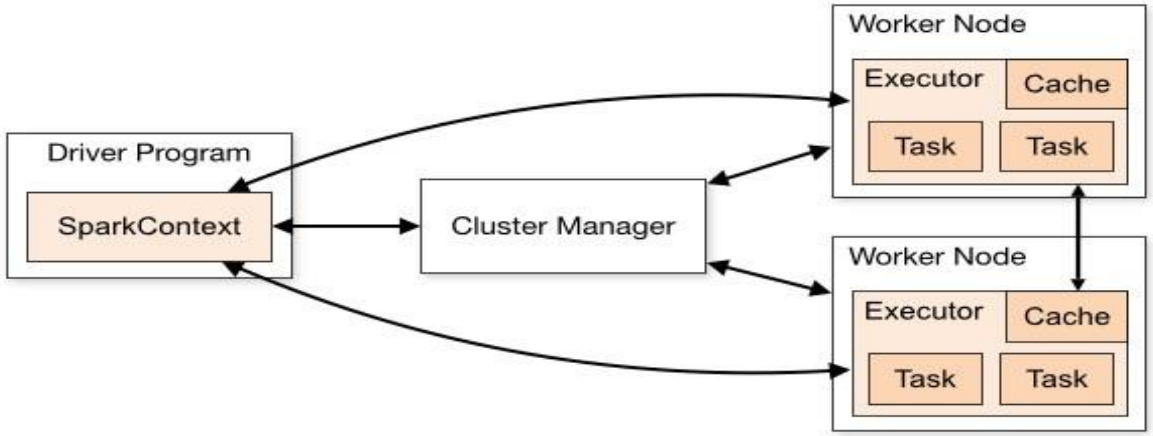
Spark rozdziela pracę między wiele maszyn, wykonuje ją równolegle i wykorzystuje pamięć operacyjną dla lepszej wydajności.

1. Driver Program - Punkt wejścia do aplikacji. Tworzy plan wykonania (DAG) i koordynuje pracę klastra

2. Cluster Manager - Zarządza zasobami klastra (CPU, RAM). Może to być Spark Standalone, YARN, Mesos, Kubernetes

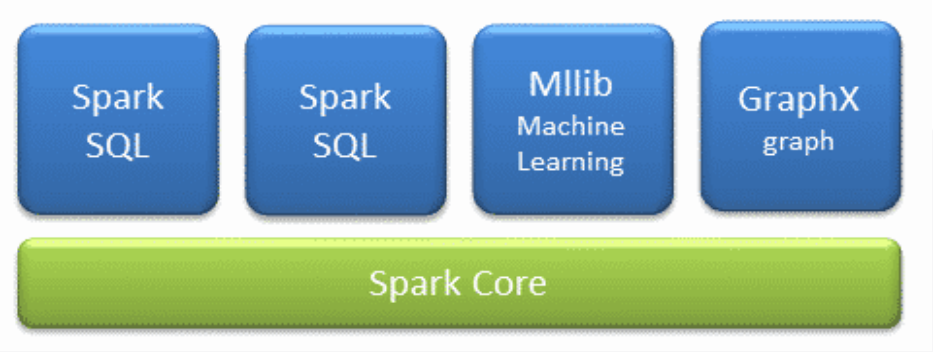
3. Executors - Procesy uruchamiane na węzłach roboczych. Wykonują zadania i przechowują dane w pamięci lub na dysku

4. Taski i RDD/DataFrame - Plan wykonania dzielony na taski rozsyłane do executorów. Dane reprezentowane jako RDD lub DataFrame (rozproszone kolekcje)



ideis
UNIWERSYTET DSW

Główne komponenty



Spark Core

– Podstawowy silnik przetwarzania danych (RDD, DAG, zarządzanie zasobami)

Spark SQL

– Obsługa danych tabelarycznych (DataFrame, SQL)

– Integracja z formatami: CSV, Parquet, JSON, JDBC

Spark Streaming

– Przetwarzanie danych strumieniowych w mikroseriach (near real-time)

– Integracja z Kafka, socketami, plikami

MLlib

– Uczenie maszynowe: klasyfikacja, regresja, klastrowanie

– Skalowalne trenowanie modeli na dużych zbiorach

GraphX

– Przetwarzanie i analiza grafów

– Operacje na wierzchołkach, krawędziach, algorytmy grafowe

ideis UNIWERSYTET DSW

Spark - RDD vs. DataFrame

RDD (Resilient Distributed Dataset) to podstawowa abstrakcja danych w Spark:

- Rozproszona, niemodyfikowalna kolekcja obiektów
- Przechowywana w pamięci lub na dysku
- Rozproszona automatycznie między węzłami klastra

DataFrame w Spark to rozproszona tabela danych z nazwanymi kolumnami

– Podobna do Pandas DataFrame lub SQL table

– Zbudowana na bazie RDD, ale oferuje wyższy poziom abstrakcji

ideis UNIWERSYTET DSW

Spark - Lazy evaluation

W PySpark obliczenia działają w modelu lazy evaluation, czyli odkładania wykonania do momentu, gdy jest to faktycznie potrzebne. Operacje takie jak filtrowanie, mapowanie czy łączenie danych nie są wykonywane od razu; PySpark jedynie buduje plan działania. Realna praca następuje dopiero wtedy, gdy wywołane zostanie działanie wymagające wyniku, na przykład zliczenie rekordów lub zapis danych.

ideis
UNIWERSYTET DSW

Spark - Transformations vs Actions

Transformations

Operacje **leniwe (lazy)**, które **tworzą nowe RDD/DataFrame**, ale **nie wyzwalają wykonania**

- Zwracają nowe, przetwarzalne obiekty
- Budują plan wykonania (DAG), ale nic się jeszcze nie dzieje

Przykłady: map, filter, select, groupBy, join

Actions

Operacje, które **wykonują obliczenia** i **zwracają wynik do drivera** lub zapisują dane

- Wyzwalają przetwarzanie zbudowanego DAG
- To one „uruchamiają” Spark

Przykłady: collect(), count(), show(), save()

ideis UNIWERSYTET DSW

Setup PySpark

Wymagania systemowe

- **Java (JDK)** w wersji 8 lub 11 (Spark działa na JVM)
- **Python** 3.x
- (Opcjonalnie) **Hadoop** lub inne środowisko klastrowe

Instalacja lokalna (najprostsza metoda)

```
pip install pyspark
```

Alternatywy uruchamiania

- **Jupyter / Colab**
- **Databricks** (online, bez konfiguracji)

ideis UNIWERSYTET DSW

PySpark - Setup Linux

Uruchamianie poleceń:

!apt-get install openjdk-17-jdk-headless -qq > /dev/null

!wget https://dlcdn.apache.org/spark/spark-3.5.7/spark-3.5.7-bin-hadoop3.tgz -O spark-3.5.7-bin-hadoop3.tgz

!tar xf spark-3.5.7-bin-hadoop3.tgz

Zestaw tych poleceń instaluje środowisko Java niezbędne do uruchomienia Sparka, pobiera archiwum z wybraną wersją Apache Spark oraz rozpakowuje je w bieżącym katalogu.

Po tym należy uruchomić w Pythonie:

import os

os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-17-openjdk-amd64"

os.environ["SPARK_HOME"] = "/content/spark-3.5.7-bin-hadoop3"

Co ustawia zmienne środowiskowe. Następnie należy zainstalować PySpark przez pip:

pip install pyspark

ideis UNIWERSYTET DSW

PySpark - Setup MacOS

Uruchamianie poleceń:

!brew install openjdk@17

!curl -LO https://dlcdn.apache.org/spark/spark-3.5.7/spark-3.5.7-bin-hadoop3.tgz

!tar xf spark-3.5.7-bin-hadoop3.tgz

Zestaw tych poleceń instaluje środowisko Java niezbędne do uruchomienia Sparka, pobiera archiwum z wybraną wersją Apache Spark oraz rozpakowuje je w bieżącym katalogu.

Po tym należy uruchomić w Pythonie:

import os

os.environ["JAVA_HOME"] = "/usr/local/opt/openjdk@17"

os.environ["SPARK_HOME"] = "/Users/TwojaNazwaUzytkownika/spark-3.5.4-bin-hadoop3"

Co ustawia zmienne środowiskowe. Następnie należy zainstalować PySpark przez pip:

pip install pyspark

ideis UNIWERSYTET DSW

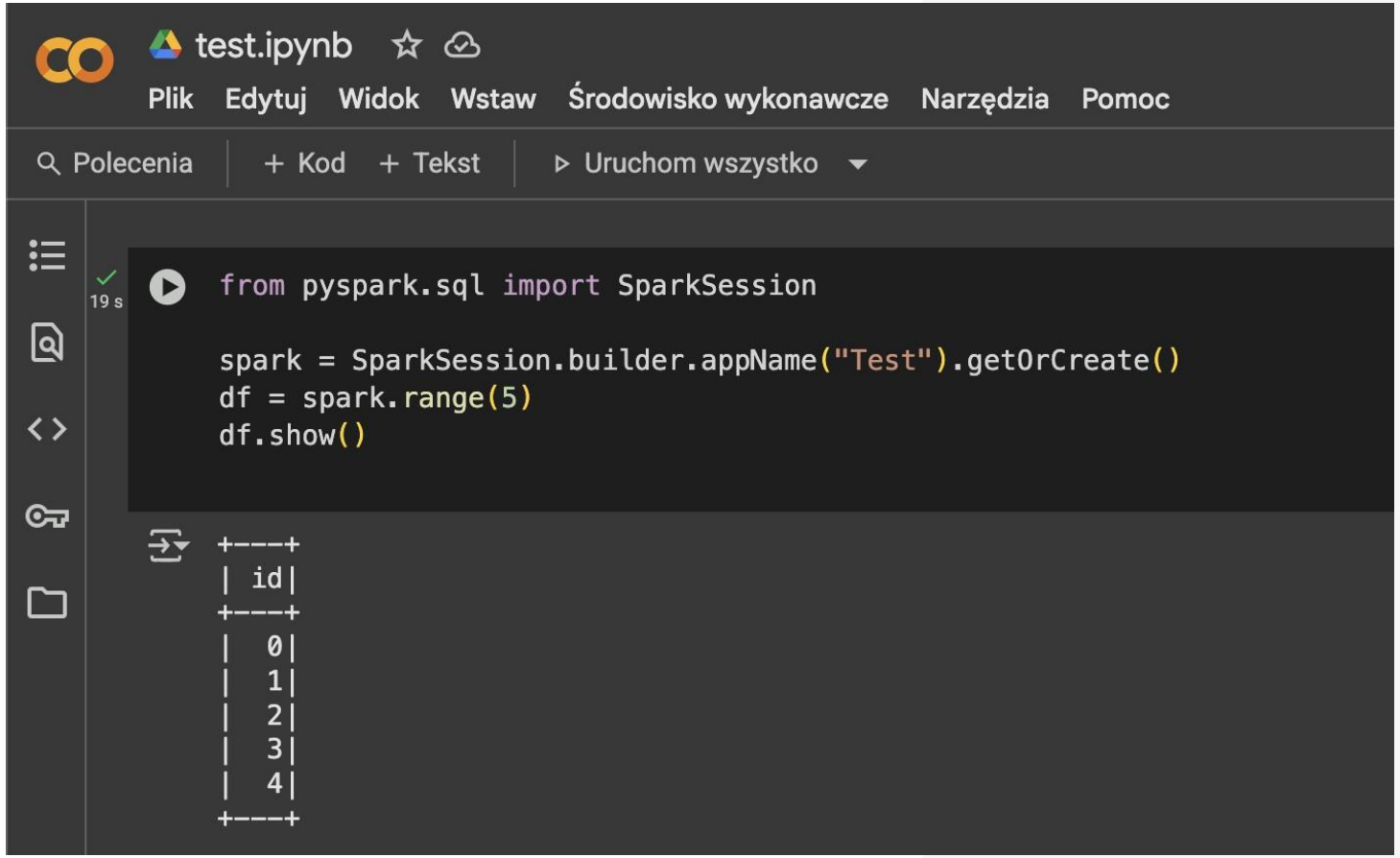
PySpark - Setup Windows

- **Instalacja Javy -** Pobierz i zainstaluj OpenJDK 17, np. ze strony https://learn.microsoft.com/en-us/java/openjdk/download
- **Pobranie Apache Spark -** Wejdź na stronę Apache Spark i pobierz wersję z wbudowanym Hadoop, np. `Spark-3.5.4-bin-hadoop3.tgz` (https://archive.apache.org/dist/spark/spark-3.5.4). Rozpakuj archiwum np. do `C:\spark\spark-3.5.4-bin-hadoop3`.
- **Ustawienie zmiennych środowiskowych** - Dodaj do `PATH` ścieżkę do folderu `bin` Sparka, np.: C:\spark\spark-3.5.4-bin-hadoop3\bin. Ustaw zmienną `PYSPARK_PYTHON=python`
- Zainstaluj PySpark przez pip: *pip install pyspark*

ideis
UNIWERSYTET DSW

PySpark w colab

PySpark działa w colab bez konieczności instalowania czegokolwiek:



ideis
UNIWERSYTET DSW

Tworzenie SparkSession

W PySpark sesja (SparkSession) to punkt startowy do pracy z danymi. Reprezentuje połączenie z silnikiem Spark i pozwala wykonywać operacje na danych w klastrze. Dzięki sesji można tworzyć DataFrame'y, wykonywać zapytania SQL, korzystać z modułów Spark SQL, MLlib czy Spark Streaming, a także kontrolować konfigurację środowiska Spark. Innymi słowy, SparkSession to „centralny hub", przez który PySpark komunikuje się z całym silnikiem Spark.

```
1   from pyspark.sql import SparkSession
2
3   spark = SparkSession.builder.appName("Tutorial").getOrCreate()
```

ideis UNIWERSYTET DSW

Spark – RDD

```
sc = spark.sparkContext

data = [1, 2, 3, 4, 5]
distData = sc.parallelize(data)
print(distData.collect())
```

✓ 0.8s

```
[1, 2, 3, 4, 5]
```

```
A = ((a, a*a) for a in range(1000))
data = sc.parallelize(A)
print(data.count())   # liczba elementów
print(data.take(10))  # pierwsze 10 elementów
```

✓ 1.5s

```
[Stage 1:>                                                          (0 + 8) / 8]
1000
[(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25), (6, 36), (7, 49), (8, 64), (9, 81)]
```

ideis UNIWERSYTET DSW

Przykładowy dataframe

```python
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, FloatType

schema = StructType([
    StructField("student_id", StringType(), True),
    StructField("name", StringType(), True),
    StructField("faculty", StringType(), True),
    StructField("program", StringType(), True),
    StructField("year", IntegerType(), True),
    StructField("gpa", FloatType(), True)
])

data = [
    ("S001", "Anna Kowalska", "KSB", "Data Science", 2, 4.5),
    ("S002", "Piotr Nowak", "UEK", "Informatyka", 1, 4.0),
    ("S003", "Marta Wiśniewska", "KSB", "Data Science", 1, 4.8),
    ("S004", "Jan Zieliński", "UEK", "Informatyka", 2, 3.9),
    ("S005", "Katarzyna Mazur", "KSB", "Data Science", 1, 4.2)
]

df_students = spark.createDataFrame(data, schema)
df_students.show()
```

✓ 0.5s

```
+----------+----------------+-------+------------+----+---+
|student_id|            name|faculty|     program|year|gpa|
+----------+----------------+-------+------------+----+---+
|      S001|   Anna Kowalska|    KSB|Data Science|   2|4.5|
|      S002|     Piotr Nowak|    UEK| Informatyka|   1|4.0|
|      S003|Marta Wiśniewska|    KSB|Data Science|   1|4.8|
|      S004|   Jan Zieliński|    UEK| Informatyka|   2|3.9|
|      S005| Katarzyna Mazur|    KSB|Data Science|   1|4.2|
+----------+----------------+-------+------------+----+---+
```

ideis UNIWERSYTET DSW

Schema w dataframe

Schema to **opis struktury DataFrame** – definiuje nazwy kolumn, typy danych i informację, czy wartości mogą być puste. Dzięki schematowi Spark wie, jak optymalnie przechowywać i przetwarzać dane.

Nazwy kolumn – identyfikatory danych w DataFrame.

Typy danych – np. StringType, IntegerType, FloatType, BooleanType.

Nullable – informacja, czy kolumna może zawierać wartości null.

Ułatwia optymalizację – Spark może lepiej planować zapytania dzięki jawnej strukturze.

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, FloatType

schema = StructType([
    StructField("student_id", StringType(), True),
    StructField("name", StringType(), True),
    StructField("faculty", StringType(), True),
    StructField("program", StringType(), True),
    StructField("year", IntegerType(), True),
    StructField("gpa", FloatType(), True)
])
```

ideis UNIWERSYTET DSW

Różne sposoby tworzenia Spark DataFrames

PySpark pozwala tworzyć DataFrame z różnych źródeł danych. Kilka przykładów:

- **Z listy słowników (Python)**

```python
data = [
    {"name": "Anna", "age": 25, "program": "Data Science"},
    {"name": "Piotr", "age": 30, "program": "Informatyka"}
]
df = spark.createDataFrame(data)
df.show()
```

✓ 0.9s

```
+---+-----+------------+
|age| name|     program|
+---+-----+------------+
| 25| Anna|Data Science|
| 30|Piotr| Informatyka|
+---+-----+------------+
```

ideis UNIWERSYTET DSW

Różne sposoby tworzenia Spark DataFrames

Z listy krotek z podaniem schematu

```python
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

schema = StructType([
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True),
    StructField("program", StringType(), True)
])

data = [("Anna", 25, "Data Science"), ("Piotr", 30, "Informatyka")]
df = spark.createDataFrame(data, schema)
df.show()
```

✓ 0.7s

```
+-----+---+------------+
| name|age|     program|
+-----+---+------------+
| Anna| 25|Data Science|
|Piotr| 30| Informatyka|
+-----+---+------------+
```

ideis UNIWERSYTET DSW

Różne sposoby tworzenia Spark DataFrames

Z plików CSV / JSON / Parquet

```python
# CSV
df_csv = spark.read.csv("data/students.csv", header=True, inferSchema=True)

# JSON
df_json = spark.read.json("data/students.json")

# Parquet
df_parquet = spark.read.parquet("data/students.parquet")
```

ideis
UNIWERSYTET DSW

Różne sposoby tworzenia Spark DataFrames

Z RDD

```
rdd = sc.parallelize([("Anna", 25, "Data Science"), ("Piotr", 30, "Informatyka")])
df = rdd.toDF(["name", "age", "program"])
df.show()
```

ideis
UNIWERSYTET DSW

Różne sposoby tworzenia Spark DataFrames

Z tabel SQL / baz danych

```
df_sql = spark.read.format("jdbc").options(
    dbtable="students",
    user="username",
    password="password"
).load()
df_sql.show()
```

ideis
UNIWERSYTET DSW

Różne sposoby tworzenia Spark DataFrames

Z danych Pandas

```python
import pandas as pd

pdf = pd.DataFrame({
    "name": ["Anna", "Piotr"],
    "age": [25, 30],
    "program": ["Data Science", "Informatyka"]
})

df = spark.createDataFrame(pdf)
df.show()
```

ideis
UNIWERSYTET DSW

Functions

PySpark oferuje moduł funkcji: pyspark.sql.functions - zbiór gotowych operacji, które można stosować na kolumnach w DataFrame.

Dzięki funkcjom można łatwo przekształcać, agregować, filtrów, formatować dane - bez potrzeby pisania skomplikowanego kodu ręcznie, operując na całych kolumnach.

```
import pyspark.sql.functions as F

✓ 0.0s
```

F to alias ułatwiający zapis funkcji wbudowanych. Większość operacji transformacyjnych i agregacyjnych znajduje się w tym module.

ideis UNIWERSYTET DSW

Wybór kolumn i aliasy

F.col odwołuje się do kolumny;
F.lit tworzy stałą;
alias zmienia nazwę kolumny.

```python
# wybór kolumn
df_select = df_students.select("student_id", "name", "gpa")


# alias i wyrażenie
df_alias = df_students.select(
F.col("student_id"),
F.col("gpa"),
F.col("name").alias("full_name"),
(F.col("gpa") * F.lit(10)).alias("gpa_x10")
)

df_alias.show()
```

✓ 0.5s

```
+----------+---+----------------+------------------+
|student_id|gpa|       full_name|           gpa_x10|
+----------+---+----------------+------------------+
|      S001|4.5|   Anna Kowalska|              45.0|
|      S002|4.0|     Piotr Nowak|              40.0|
|      S003|4.8|Marta Wiśniewska| 48.00000190734863|
|      S004|3.9|   Jan Zieliński|39.000000953674316|
|      S005|4.2| Katarzyna Mazur| 41.99999809265137|
+----------+---+----------------+------------------+
```

ideis
UNIWERSYTET DSW

Tworzenie nowych kolumn - withColumn

withColumn tworzy nową kolumnę na podstawie wyrażenia Spark.

```
# dodać kolumnę year_plus_one
df_new = df_students.withColumn("year_plus_one", F.col("year") + F.lit(1))


# zaokrąglenie GPA do 1 miejsca po przecinku
df_round = df_students.withColumn("gpa_round", F.round(F.col("gpa"), 1))
df_round.show()
```

✓ 0.5s

```
+----------+----------------+-------+------------+----+---+---------+
|student_id|            name|faculty|     program|year|gpa|gpa_round|
+----------+----------------+-------+------------+----+---+---------+
|      S001|   Anna Kowalska|    KSB|Data Science|   2|4.5|      4.5|
|      S002|     Piotr Nowak|    UEK| Informatyka|   1|4.0|      4.0|
|      S003|Marta Wiśniewska|    KSB|Data Science|   1|4.8|      4.8|
|      S004|   Jan Zieliński|    UEK| Informatyka|   2|3.9|      3.9|
|      S005| Katarzyna Mazur|    KSB|Data Science|   1|4.2|      4.2|
+----------+----------------+-------+------------+----+---+---------+
```

ideis UNIWERSYTET DSW

Warunkowe wartości - when / otherwise

```python
df_grade = df_students.withColumn(
    "gpa_level",
    F.when(F.col("gpa") >= 4.5, F.lit("very_good"))
    .when(F.col("gpa") >= 4.0, F.lit("good"))
    .otherwise(F.lit("needs_improvement"))
)

df_grade.show()
```

✓ 0.6s

```
+----------+----------------+-------+------------+----+---+-----------------+
|student_id|            name|faculty|     program|year|gpa|        gpa_level|
+----------+----------------+-------+------------+----+---+-----------------+
|      S001|   Anna Kowalska|    KSB|Data Science|   2|4.5|        very_good|
|      S002|     Piotr Nowak|    UEK| Informatyka|   1|4.0|             good|
|      S003|Marta Wiśniewska|    KSB|Data Science|   1|4.8|        very_good|
|      S004|   Jan Zieliński|    UEK| Informatyka|   2|3.9|needs_improvement|
|      S005| Katarzyna Mazur|    KSB|Data Science|   1|4.2|             good|
+----------+----------------+-------+------------+----+---+-----------------+
```

ideis UNIWERSYTET DSW

Łączenie tekstu i ekstrakcja

```python
# wydzielenie nazwiska (zakładając format: Imię Nazwisko)
df_lastname = df_students.withColumn(
    "lastname",
    F.split(F.col("name"), " ")[1]
)

df_lastname.show()
```

✓ 0.4s

```
+----------+----------------+-------+------------+----+---+----------+
|student_id|            name|faculty|     program|year|gpa|  lastname|
+----------+----------------+-------+------------+----+---+----------+
|      S001|   Anna Kowalska|    KSB|Data Science|   2|4.5|  Kowalska|
|      S002|     Piotr Nowak|    UEK| Informatyka|   1|4.0|     Nowak|
|      S003|Marta Wiśniewska|    KSB|Data Science|   1|4.8|Wiśniewska|
|      S004|   Jan Zieliński|    UEK| Informatyka|   2|3.9| Zieliński|
|      S005| Katarzyna Mazur|    KSB|Data Science|   1|4.2|     Mazur|
+----------+----------------+-------+------------+----+---+----------+
```

```python
# skrócony id: faculty-program
df_id = df_students.withColumn(
    "short_id",
    F.concat_ws("-", F.col("faculty"), F.col("program"))
)

df_id.show()
```

✓ 0.5s

```
+----------+----------------+-------+------------+----+---+----------------+
|student_id|            name|faculty|     program|year|gpa|        short_id|
+----------+----------------+-------+------------+----+---+----------------+
|      S001|   Anna Kowalska|    KSB|Data Science|   2|4.5|KSB-Data Science|
|      S002|     Piotr Nowak|    UEK| Informatyka|   1|4.0| UEK-Informatyka|
|      S003|Marta Wiśniewska|    KSB|Data Science|   1|4.8|KSB-Data Science|
|      S004|   Jan Zieliński|    UEK| Informatyka|   2|3.9| UEK-Informatyka|
|      S005| Katarzyna Mazur|    KSB|Data Science|   1|4.2|KSB-Data Science|
+----------+----------------+-------+------------+----+---+----------------+
```

ideis UNIWERSYTET DSW

Agregacje i grupowanie

```python
agg_df = (
    df_students
    .groupBy("program")
    .agg(
        F.count("student_id").alias("count_students"),
        F.round(F.avg("gpa"), 2).alias("avg_gpa"),
        F.min("gpa").alias("min_gpa"),
        F.max("gpa").alias("max_gpa")
    )
)

agg_df.show()
```

✓ 1.2s

```
+------------+--------------+-------+-------+-------+
|     program|count_students|avg_gpa|min_gpa|max_gpa|
+------------+--------------+-------+-------+-------+
|Data Science|             3|    4.5|    4.2|    4.8|
| Informatyka|             2|   3.95|    3.9|    4.0|
+------------+--------------+-------+-------+-------+
```

ideis UNIWERSYTET DSW

Operacje okienkowe (Window)

```
from pyspark.sql import Window


w = Window.partitionBy("program").orderBy(F.col("gpa").desc())

ranked = df_students.withColumn("rank_in_program", F.dense_rank().over(w))
ranked.show()
```

✓ 0.7s

```
+----------+----------------+-------+------------+----+---+---------------+
|student_id|            name|faculty|     program|year|gpa|rank_in_program|
+----------+----------------+-------+------------+----+---+---------------+
|      S003|Marta Wiśniewska|    KSB|Data Science|   1|4.8|              1|
|      S001|   Anna Kowalska|    KSB|Data Science|   2|4.5|              2|
|      S005| Katarzyna Mazur|    KSB|Data Science|   1|4.2|              3|
|      S002|     Piotr Nowak|    UEK| Informatyka|   1|4.0|              1|
|      S004|   Jan Zieliński|    UEK| Informatyka|   2|3.9|              2|
+----------+----------------+-------+------------+----+---+---------------+
```

ideis UNIWERSYTET DSW

Obsługa braków

```python
# wypełnienie domyślną wartością
df_filled = df_students.fillna({"gpa": 0.0, "faculty": "unknown"})

df_filled.show()
```

✓ 0.6s

```
+----------+----------------+-------+------------+----+---+
|student_id|            name|faculty|     program|year|gpa|
+----------+----------------+-------+------------+----+---+
|      S001|   Anna Kowalska|    KSB|Data Science|   2|4.5|
|      S002|     Piotr Nowak|    UEK| Informatyka|   1|4.0|
|      S003|Marta Wiśniewska|    KSB|Data Science|   1|4.8|
|      S004|   Jan Zieliński|    UEK| Informatyka|   2|3.9|
|      S005| Katarzyna Mazur|    KSB|Data Science|   1|4.2|
+----------+----------------+-------+------------+----+---+
```

ideis UNIWERSYTET DSW

Obsługa typów danych

```python
# rzutowanie typu kolumny
df_cast = df_students.withColumn("year_str", F.col("year").cast("string"))

df_cast.show()
print(df_cast.dtypes)
```

✓ 0.4s

```
+----------+----------------+-------+------------+----+---+--------+
|student_id|            name|faculty|     program|year|gpa|year_str|
+----------+----------------+-------+------------+----+---+--------+
|      S001|   Anna Kowalska|    KSB|Data Science|   2|4.5|       2|
|      S002|     Piotr Nowak|    UEK| Informatyka|   1|4.0|       1|
|      S003|Marta Wiśniewska|    KSB|Data Science|   1|4.8|       1|
|      S004|   Jan Zieliński|    UEK| Informatyka|   2|3.9|       2|
|      S005| Katarzyna Mazur|    KSB|Data Science|   1|4.2|       1|
+----------+----------------+-------+------------+----+---+--------+

[('student_id', 'string'), ('name', 'string'), ('faculty', 'string'), ('program', 'string'), ('year', 'int'), ('gpa', 'float'), ('year_str', 'string')]
```

ideis
UNIWERSYTET DSW

Sortowanie i pivot

```
df.orderBy(df.passengers.desc()).show(5)
```

year	month	passengers	traffic_level
1960	July	622	high
1960	August	606	high
1959	August	559	high
1959	July	548	high
1960	June	535	high

only showing top 5 rows

```
pivot = df.groupBy("month").pivot("year").sum("passengers")
pivot.show()
```

month	1949	1950	1951	1952	1953	1954	1955	1956	1957	1958	1959	1960
July	148	170	199	230	264	302	364	413	465	491	548	622
November	104	114	146	172	180	203	237	271	305	310	362	390
February	118	126	150	180	196	188	233	277	301	318	342	391
January	112	115	145	171	196	204	242	284	315	340	360	417
March	132	141	178	193	236	235	267	317	356	362	406	419
October	119	133	162	191	211	229	274	306	347	359	407	461
May	121	125	172	183	229	234	270	318	355	363	420	472
August	148	170	199	242	272	293	347	405	467	505	559	606
April	129	135	163	181	235	227	269	313	348	348	396	461
June	135	149	178	218	243	264	315	374	422	435	472	535
December	118	140	166	194	201	229	278	306	336	337	405	432
September	136	158	184	209	237	259	312	355	404	404	463	508

Porządkowanie, ograniczanie i zapisywanie

```
# sortowanie po gpa malejąco
df_sorted = df_students.orderBy(F.col("gpa").desc())


# top 3 studentów
df_top3 = df_sorted.limit(3)


# zapis do parquet
df_top3.write.mode("overwrite").parquet("students")
```

ideis
UNIWERSYTET DSW

Join i zapis do parquet

```
from pyspark.sql import Row

meta = spark.createDataFrame([
    Row(month="January", season="Winter"),
    Row(month="July", season="Summer")
])

df_joined = df.join(meta, on="month", how="left")
df_joined.show()
```

```
df.write.mode("overwrite").parquet("flights_cleaned.parquet")
```

ideis
UNIWERSYTET DSW

Spark SQL

Uwagi:

- Można pisać pełne zapytania SQL.
- Zapytania SQL i operacje DataFrame są równoważne i mogą być mieszane.
- Widoki tymczasowe są przechowywane w ramach sesji Spark.

```
# Rejestracja DataFrame jako widoku:
df_students.createOrReplaceTempView("students")

# Przykład prostego zapytania:
spark.sql("SELECT name, gpa FROM students WHERE gpa > 4.0").show()

# Przykład agregacji:
spark.sql(
    "SELECT program, COUNT(*) AS count_students, ROUND(AVG(gpa), 2) AS avg_gpa "
    "FROM students GROUP BY program"
).show()

# Przykład sortowania i ograniczania wyników:
spark.sql(
    "SELECT name, gpa FROM students ORDER BY gpa DESC LIMIT 3"
).show()
```

ideis
UNIWERSYTET DSW

UDF – User Defined Functions

UDF (User Defined Function) pozwala używać własnych funkcji Pythona w kolumnach DataFrame.

Uwagi:

- UDF-y są **wolniejsze** niż funkcje wbudowane (lower(), concat(), when(), itp.)
- Nie są automatycznie optymalizowane przez Catalyst (engine Sparka)
- Nie wspierają w pełni null-safeoperacji → warto dodawać if/ try

Kiedy używać UDF?

- Gdy potrzebujesz logiki, której Spark nie oferuje „w pudełku”
- Gdy logika jest specyficzna i trudna do wyrażenia SQL-em

Kiedy unikać?

- W krytycznych pipeline’ach — preferuj funkcje Spark SQL

```python
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

@udf(StringType())
def uppercase(text):
    return text.upper() if text else None

df.withColumn("month_upper", uppercase("month")).show()
```

ideis UNIWERSYTET DSW

explain

Metoda .explain()pokazuje **plan wykonania zapytania** (DAG) wygenerowany przez Catalyst – optymalizator zapytań w Spark SQL.

```python
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Tutorial").getOrCreate()

df = spark.read.csv("flights.csv", header=True, inferSchema=True)

df_filtered = df.filter(df.passengers > 300)
df_grouped = df_filtered.groupBy("year").count()

df_grouped.explain()
```

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[year#16], functions=[count(1)])
   +- Exchange hashpartitioning(year#16, 200), ENSURE_REQUIREMENTS, [id=#34]
      +- HashAggregate(keys=[year#16], functions=[partial_count(1)])
         +- Project [year#16]
            +- Filter (isnotnull(passengers#18) AND (passengers#18 > 300))
               +- FileScan csv [year#16,passengers#18] Batched: false, DataFilters: [isnotnull(passengers#18), (passengers#18 > 300)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Use
rs/przemyslawniedziela/Desktop/class_files/aap/flights.csv], PartitionFilters: [], PushedFilters: [IsNotNull(passengers), GreaterThan(passengers,300)], ReadSchema: struct<year:int,passengers:int
>
```

- Pozwala zrozumieć **gdzie pojawia się shuffle**
- Pokazuje **czy Spark używa broadcast join**
- Umożliwia diagnozę wolnych zapytań
- Pomaga pisać wydajny kod Spark

ideis UNIWERSYTET DSW

Cache, persist

Domyślnie Spark oblicza wszystko od nowa przy każdym użyciu DataFrame. Użycie cache() lub persist() pozwala zachować dane w pamięci i przyspieszyć kolejne operacje.

```
df_filtered = df.filter(df.passengers > 300).cache()

df_filtered.count()      # trigger — dane są obliczane i zapamiętane
df_filtered.groupBy("year").avg("passengers").show()  # szybciej!
```

Kiedy używać?

Gdy korzystasz wielokrotnie z tych samych danych

Gdy dane są drogie w obliczeniu (join, agregacja)

cache()to skrót dla persist()z domyślną opcją MEMORY_AND_DISK– czyli przechowuje dane w pamięci, a jeśli się nie zmieszczą, zapisuje je na dysk.

ideis UNIWERSYTET DSW

partycjonowanie

Co to jest partycja?

- Jednostka logiczna danych w Spark – to **najmniejszy fragment przetwarzany przez executor**
- Każdy DataFrameskłada się z wielu partycji (domyślnie np. 200)
- Partycje decydują o **równoległości** i **wydajności**

Możemy sprawdzać liczbę partycji, zmieniać ją i ustawiać partycje na zmiennych.

Wskazówki:

- **Zwiększ** partycje przy dużych wejściach i joinach (repartition)
- **Zmniejsz** partycje przy zapisie (coalesce)
- Używaj **partitionBy()** dla danych historycznych, logów, itp.

ideis UNIWERSYTET DSW

shuffling

Shuffling = proces **przenoszenia danych pomiędzy partycjami** (i węzłami) podczas operacji, które tego wymagają.

Kiedy występuje?

- `join()` — gdy łączysz dane między partycjami
- `groupBy()` — dane muszą trafić na te same executory
- `distinct()` i `repartition()`
- `orderBy()` — wymaga pełnego sortowania

Skutki shufflingu:

- Wolniejsze działanie (sieć + dysk)
- Większe zużycie pamięci
- Potencjalne przeciążenie executora / OOM

ideis UNIWERSYTET DSW

Triki -> Broadcast join

Broadcast Join to sposób na rozsłanie małej tabeli do wszystkich węzłów, żeby uniknąć kosztownego shuffle'a (przesyłania danych po sieci).

```
from pyspark.sql.functions import broadcast

df_large = spark.read.csv("large.csv", header=True, inferSchema=True)
df_small = spark.read.csv("small.csv", header=True, inferSchema=True)

df_joined = df_large.join(broadcast(df_small), on="key", how="inner")
```

Kiedy używać?

- Gdy **jedna z tabel jest mała** (np. < 10 MB)
- Gdy chcesz uniknąć kosztownego shuffle join

ideis UNIWERSYTET DSW

Inne triki

https://medium.com/@sounder.rahul/hands-on-coding-pyspark-optimization-techniques-construction-domain-data-6508cb822bf9

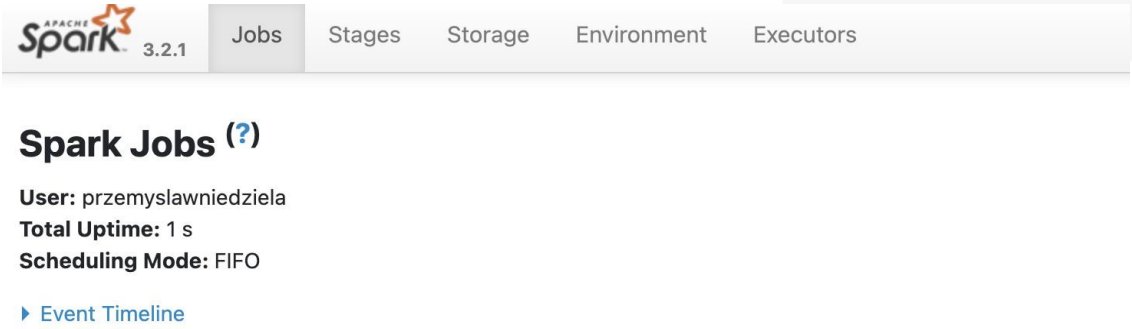
https://medium.com/@sounder.rahul/pyspark-optimization-techniques-for-data-engineers-df5033778709

https://community.databricks.com/t5/data-engineering/pyspark-optimizations-and-best-practices/td-p/10456

ideis
UNIWERSYTET DSW

Spark UI

Spark UI to wbudowany interfejs webowy, który pokazuje co dokładnie dzieje się w tle: zadania, etapy, czas wykonania, użycie pamięci, shuffling, partycjonowanie itd.



Jak otworzyć? Działa automatycznie przy lokalnym uruchomieniu Sparka. Otwórz w przeglądarce: http://localhost:4040

Po co zaglądać?

Diagnostyka problemów z wydajnością

Analiza kosztów shuffle

Sprawdzanie czy działa broadcast join, cache, itd.

ideis UNIWERSYTET DSW

DEMO

Wykonanie prostych analizach na podstawie danych Otomoto (źródło danych).

```
# Średnia cena i przebieg per marka i model
df.groupBy("vehicle_brand", "vehicle_model") \
  .agg(
      F.round(F.avg("price_num"), 2).alias("avg_price"),
      F.round(F.avg("mileage_km"), 2).alias("avg_mileage")
  ) \
  .orderBy(F.col("avg_price").desc()) \
  .show(20, truncate=False)
```

vehicle_brand	vehicle_model	avg_price	avg_mileage
Ferrari	812 GTS	2750000.0	2000.0
Mercedes-Benz	SLR	2340000.0	2380.0
Ferrari	SF90 Stradale	2293626.0	1855.75
Rolls-Royce	Dawn	1999999.0	4300.0
Lamborghini	Aventador	1822661.67	14467.5
Ferrari	812 Superfast	1670200.0	16414.0
Lamborghini	Murcielago	1550000.0	46950.0
McLaren	Artura	1499000.0	2100.0
McLaren	675Lt	1290000.0	18500.0
McLaren	600lt-coupe	1289000.0	8500.0
Lamborghini	Huracan	1258737.0	25671.46
Rolls-Royce	Phantom	1225000.0	22000.0
Rolls-Royce	Ghost	1206316.67	39451.67
Bentley	Bentayga	1200222.11	34022.72
Lamborghini	Diablo	1199900.0	32164.0
Rolls-Royce	Wraith	1183000.0	54333.33
Ferrari	GTC4Lusso	1170650.0	39061.6
Mercedes-Benz	SLS	1123316.67	24710.5
Lamborghini	Urus	1114100.0	23372.25
McLaren	altul	1099000.0	16300.0

ideis UNIWERSYTET DSW

Zadanie

Na zbiorze "Chicago_Crimes" wykonaj czyszczenie danych (usuwanie duplikatów, braków, błędnych dat). Dodaj kolumnę klasyfikującą porę dnia (np. noc, dzień, wieczór) za pomocą UDF oraz zoptymalizuj przetwarzanie przy użyciu `cache()`, `broadcast join` i partycjonowania po roku. Zapisz wynikowe dane do formatu Parquet z podziałem na partycje.

Przeprowadź analizę statystyczną przestępstw według typu, lokalizacji i czasu. Zapytania opisz poprzez `.explain()`.

Opcjonalnie: Wykorzystaj MLlib do zbudowania modelu klasyfikującego typ przestępstwa na podstawie godziny, lokalizacji i innych atrybutów.

ideis
UNIWERSYTET DSW